Session-Layer Protocols for IoT: HTTP, CoAP and XMPP

Communication Technologies for the Internet of Things, Lecture 11

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- The place of session and application protocols in the IoT communication stack
- HTTP: model, characteristics and limitations on constrained devices
- CoAP: motivation, architecture and features
- CoAP: message format, reliability and congestion control
- XMPP: federated architecture and presence model

Agenda (2/2)

- Comparison of the three protocols
- Available implementations

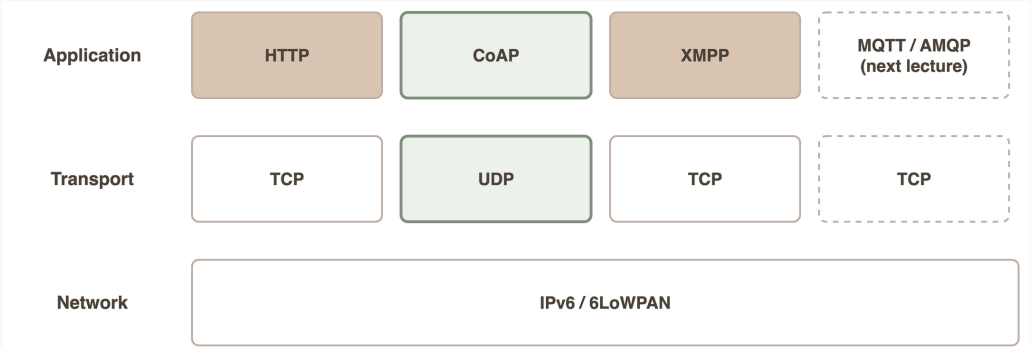
Place in the Communication Stack (1/2)

- The previous lectures covered the lower layers: medium access, network and routing.
- Once the problem of getting bytes from one node to another is solved, what remains is to define how applications structure the information they exchange.
- This is the role of the protocols covered in this lecture, placed in the session and application layers.
- The choice of protocol determines the available interaction model: request-response, publish-subscribe, or asynchronous message exchange.

Place in the Communication Stack (2/2)

- It also determines the cost in bandwidth, memory and energy, decisive factors when the device is constrained.
- This lecture covers direct-interaction protocols; the next lecture covers message-oriented middleware.

The Protocols in the Stack



The protocols in this lecture define how applications structure information

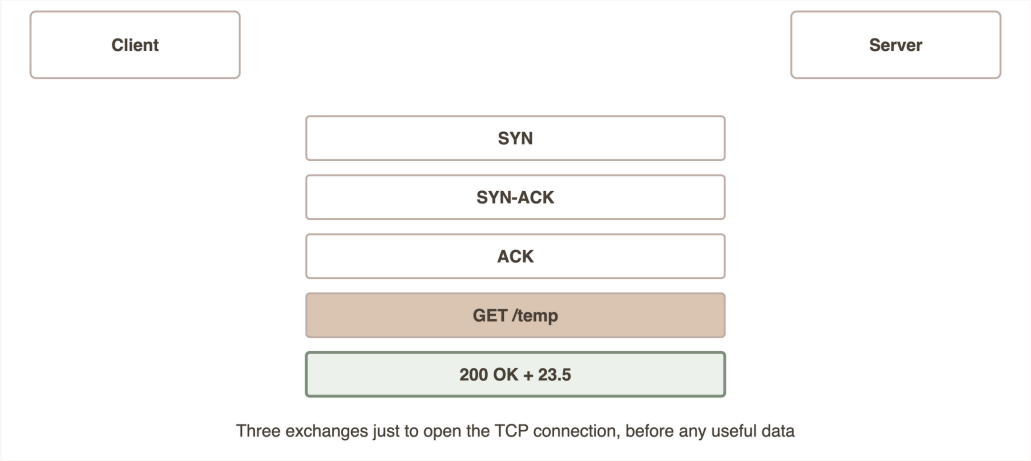
HTTP: Model and Characteristics (1/2)

- HTTP is based on a request-response model: the client sends a request and the server returns a response, without the server taking the initiative of the communication.
- The main methods are GET, POST, PUT and DELETE, which express operations on resources identified by URI.
- It is a stateless protocol: each request is independent of the previous ones, which simplifies the server but forces context to be retransmitted.
- It runs over TCP, which guarantees reliable, ordered delivery, at the cost of the three-way connection setup and of keeping state per connection.

HTTP: Model and Characteristics (2/2)

- Headers are textual and verbose, which makes diagnosis easier but significantly increases the volume of data transmitted.
- The REST model, built on HTTP, became the de facto standard for web service interfaces.

An HTTP Request over TCP



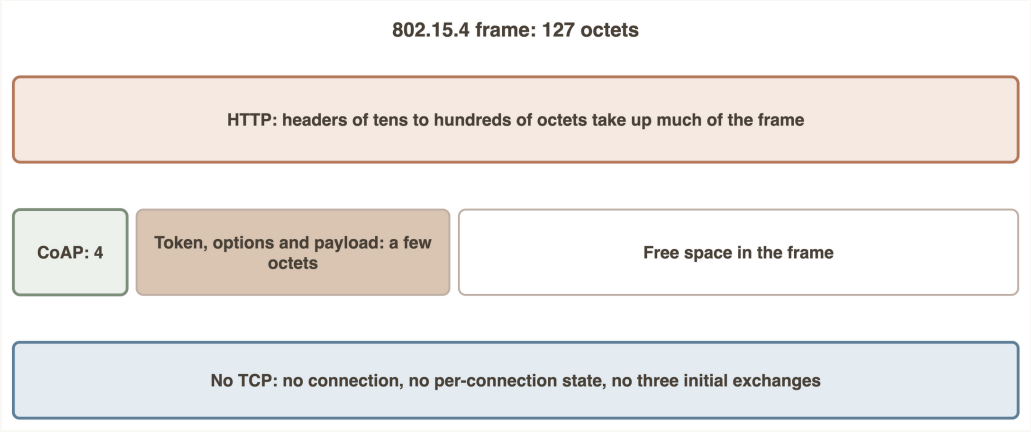
Why HTTP Is Unsuitable for Constrained Devices (1/2)

- The size of textual headers is out of proportion to the payload typical of a sensor, which rarely exceeds a few tens of bytes.
- Recall the problem already identified in 6LoWPAN: in a 127-octet IEEE 802.15.4 frame, there is no room for HTTP headers.
- TCP connection setup requires three message exchanges before any useful data is transmitted, with a relevant energy cost.
- Keeping state per connection consumes memory, a scarce resource on microcontrollers.

Why HTTP Is Unsuitable for Constrained Devices (2/2)

- The request-response model forces the client to poll the device periodically, even when nothing has changed.
- HTTP does not support broadcast to multiple recipients, a feature useful when configuring several devices at once.

The Cost of Headers



CoAP: Constrained Application Protocol (1/2)

- Defined by the IETF as "a specialised web transfer protocol for use with constrained nodes and constrained networks in the Internet of Things".
- Developed by the IETF CoRE working group, with proposed standard status in RFC 7252.
- It can be described as a light and fast version of HTTP, designed to handle simple resources in networks of constrained nodes.
- It provides a request-response interaction model similar to that of HTTP on the Web.

CoAP: Constrained Application Protocol (2/2)

- This deliberate similarity eases integration between sensor networks and existing web applications, through translation proxies.
- Architecturally, HTTP is coded according to the REST style, while CoAP is coded according to CoRE, the Constrained RESTful Environment.

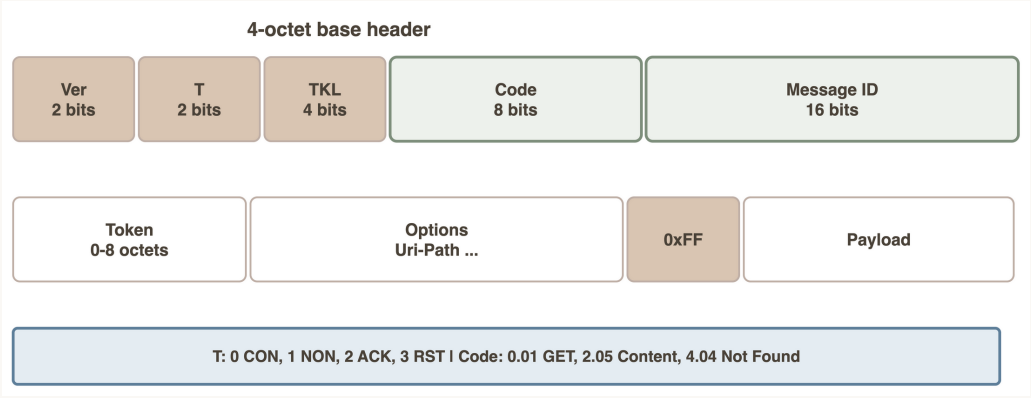
CoAP: Features (1/2)

- It uses Uniform Resource Identifiers (URI) to access resources, just like HTTP.
- It supports the GET, POST, PUT and DELETE methods, with semantics equivalent to those of HTTP.
- It supports content type indication, allowing formats such as XML, JSON and CBOR, the latter especially compact and suited to constrained networks.
- It integrates resource discovery through the `.well-known/core` path, implemented according to the link format defined by CoRE.

CoAP: Features (2/2)

- It supports intermediaries and caching: responses can be stored for faster replies, and proxies can be used to reduce traffic.
- It supports multicast communication, a possibility that follows from the use of UDP as transport.
- It allows asynchronous message exchange and was designed to be extensible.

The CoAP Message



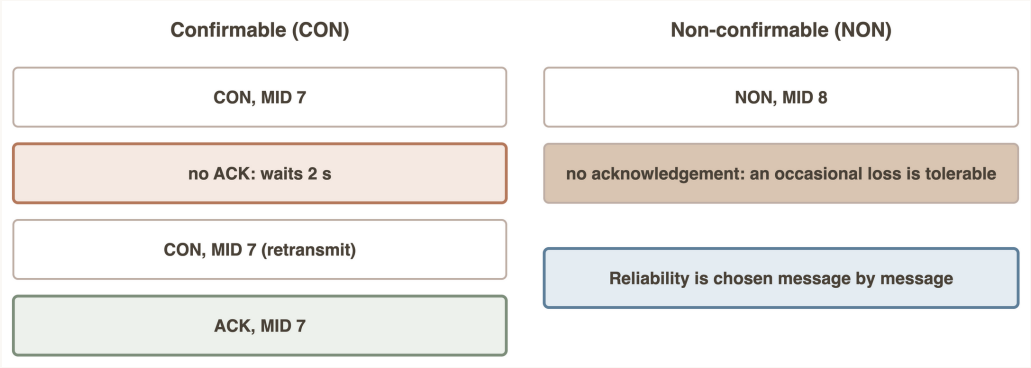
CoAP: Reliability over UDP (1/2)

- CoAP runs over UDP, a protocol that offers no guarantees of delivery or ordering.
- Reliability is therefore ensured at the application level, and not by the transport.
- **Confirmable messages:** require acknowledgement from the recipient; in the absence of a reply, the sender retransmits with exponential back-off.
- **Non-confirmable messages:** are sent with no expectation of acknowledgement, suited to periodic readings where occasional loss is tolerable.

CoAP: Reliability over UDP (2/2)

- When faced with a failure or a message that cannot be processed, the recipient replies with a reset message.
- The protocol includes congestion control mechanisms, needed because UDP does not provide them.
- This architecture lets the programmer choose, message by message, the trade-off between reliability and consumption.

Confirmable and Non-Confirmable



CoAP: Signalling Reduction Compared with HTTP (1/2)

- Replacing TCP with UDP removes the three-way connection setup and the associated state.
- Binary header encoding, instead of the textual encoding of HTTP, drastically reduces the volume transmitted.
- The CoAP base header takes only 4 bytes, a value to compare with the hundreds of bytes typical of an HTTP header.
- Caching and the use of proxies reduce the number of interactions that cross the constrained network.

CoAP: Signalling Reduction Compared with HTTP (2/2)

- Multicast support makes it possible to reach several devices with a single transmission.
- The combined result is a very significant reduction in signalling, which translates directly into lower energy consumption and greater autonomy.

XMPP: Extensible Messaging and Presence Protocol (1/2)

- A protocol standardised by the IETF, defined in RFC 6120, originating in the domain of instant messaging.
- It is based on the exchange of XML fragments, called *stanzas*, over persistent TCP connections.
- The architecture is federated, like e-mail: each domain runs its own server, and the servers talk to each other.
- Participants are identified by an address in the format user@domain, which dispenses with a central registry.

XMPP: Extensible Messaging and Presence Protocol (2/2)

- The concept of presence is native to the protocol: each entity can announce its state, and those interested are notified of changes.
- The XEP-0060 extension adds publish-subscribe, allowing the same data distribution model found in message middleware.

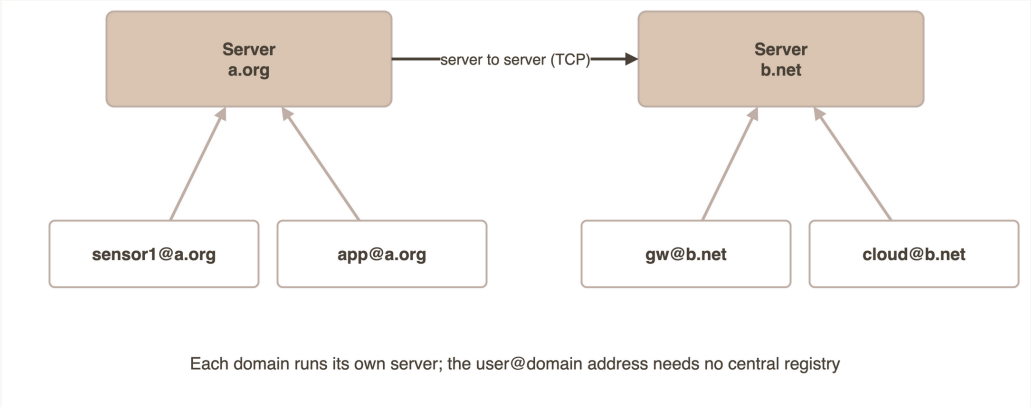
XMPP in an IoT Context (1/2)

- The federated architecture is its main advantage: it allows installations from different organisations to be interconnected without depending on a common central infrastructure.
- The presence model applies naturally to devices, by explicitly signalling whether a sensor is available or not.
- XML encoding is, however, verbose, which is a significant disadvantage in constrained networks.
- The need to keep persistent TCP connections is incompatible with devices that spend most of their time asleep.

XMPP in an IoT Context (2/2)

- In practice, XMPP is better suited to the gateway and applications side than to the sensor side.
- It is the option to consider when interoperability between different administrative domains is a central requirement of the solution.

XMPP Federation



Comparing the Three Protocols (1/2)

- **Transport:** HTTP and XMPP run over TCP; CoAP runs over UDP, with reliability handled in the application.
- **Interaction model:** HTTP and CoAP follow request-response; XMPP favours asynchronous message exchange and presence, with publish-subscribe by extension.
- **Encoding:** HTTP uses text, XMPP uses XML, and CoAP uses compact binary encoding.
- **Protocol weight:** CoAP is the lightest, followed by HTTP, with XMPP the heaviest because of the verbosity of XML.

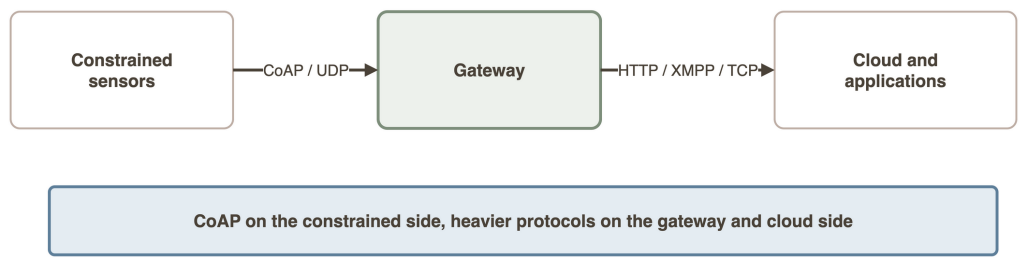
Comparing the Three Protocols (2/2)

- **Multicast:** only CoAP supports multicast natively.
- **Suitability for constrained devices:** CoAP was designed for this purpose; HTTP and XMPP are preferable on the gateway and application side.
- In a complete architecture, it is common for them to coexist: CoAP between sensors and gateway, HTTP or XMPP between gateway and cloud services.

The Three Protocols in a Table

	HTTP	CoAP	XMPP
Transport	TCP	UDP	TCP
Model	request-response	request-response	messages and presence
Encoding	text	binary	XML
Multicast	no	yes	no
Weight	medium	light	heavy

A Complete Architecture



CoAP Implementations (1/2)

- **Constrained devices, typically in C:** Erbium, libcoap, tinydtls, SMCP, microcoap, cantcoap and Lobaro CoAP.
- **Java:** Californium, nCoAP and Leshan, the latter aimed at device management.
- **C#:** CoAP.NET, CoAPSharp and Waher.Networking.CoAP.
- **Python:** txThings.
- **Android:** Californium and nCoAP; the Aneska application is a simple CoAP browser based on txThings.

CoAP Implementations (2/2)

- **Commercial hardware:** the IKEA Trådfri range uses CoAP to communicate with its devices, as do Lobaro solutions over 6LoWPAN and the Genie IoT 1200 and 2400 series.
- Reference information is available at coap.technology and in RFC 7252; the Copper extension for Firefox lets CoAP resources be explored interactively.

Lecture Summary (1/2)

- Session and application protocols define how applications structure the information they exchange, once the transport problem is solved.
- HTTP is the reference protocol on the Web, but the verbosity of its headers, the cost of TCP and the periodic polling model make it unsuitable for constrained devices.
- CoAP deliberately keeps the REST semantics of HTTP, but replaces the transport with UDP, adopts binary encoding and adds resource discovery, caching and multicast.

Lecture Summary (2/2)

- Reliability in CoAP is handled in the application, through the distinction between confirmable and non-confirmable messages, which allows the trade-off to be chosen message by message.
- XMPP offers a federated architecture and a native presence model, valuable in interconnecting organisations, but is too heavy for the sensor side.
- The next lecture covers message-oriented middleware, which solves the complementary problem: decoupling who produces data from who consumes it.

Thank you.

Questions and comments